Mesos

Overview

- Rapid innovation in distributed/cloud computing
- No single framework optimal for all applications
- Running each framework on its dedicated cluster
  - Unable to take advantage of idle resources between frameworks
  - Hard to share data (input or output)
- Would like to run multiple frameworks on the same cluster

Monolithic Scheduling

- A scheduler may have a number of inputs:

1. Job requirements
   - Response time
   - Throughput
   - Availability
2. Job execution plan
   - Task DAG
   - Inputs/Outputs
3. Estimates
   - Task duration
   - Input sizes
   - Transfer sizes

Monolithic Scheduling

- Advantages: as complete state known, “optimal” decisions can be made
- Disadvantages: complexity (scalability/resilience), incorporating additional frameworks can be difficult

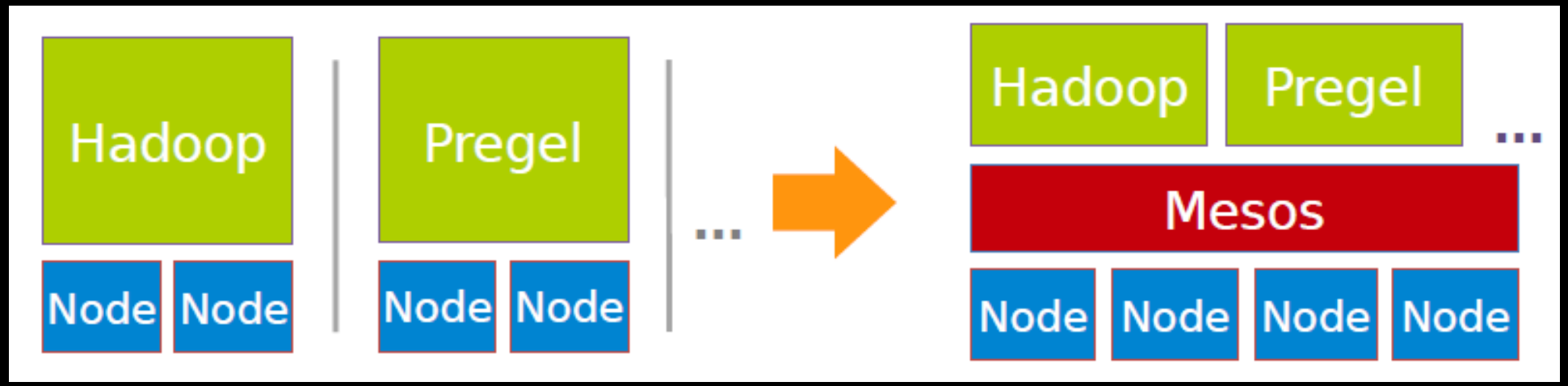
Two-Level Scheduler

- Top-level Mesos Scheduler
  1. Input: Organization policies
  2. Input: Resource availability
  3. Output: Framework Schedule
- Framework Scheduler
  1. Input: Framework Schedule
  2. Task schedule

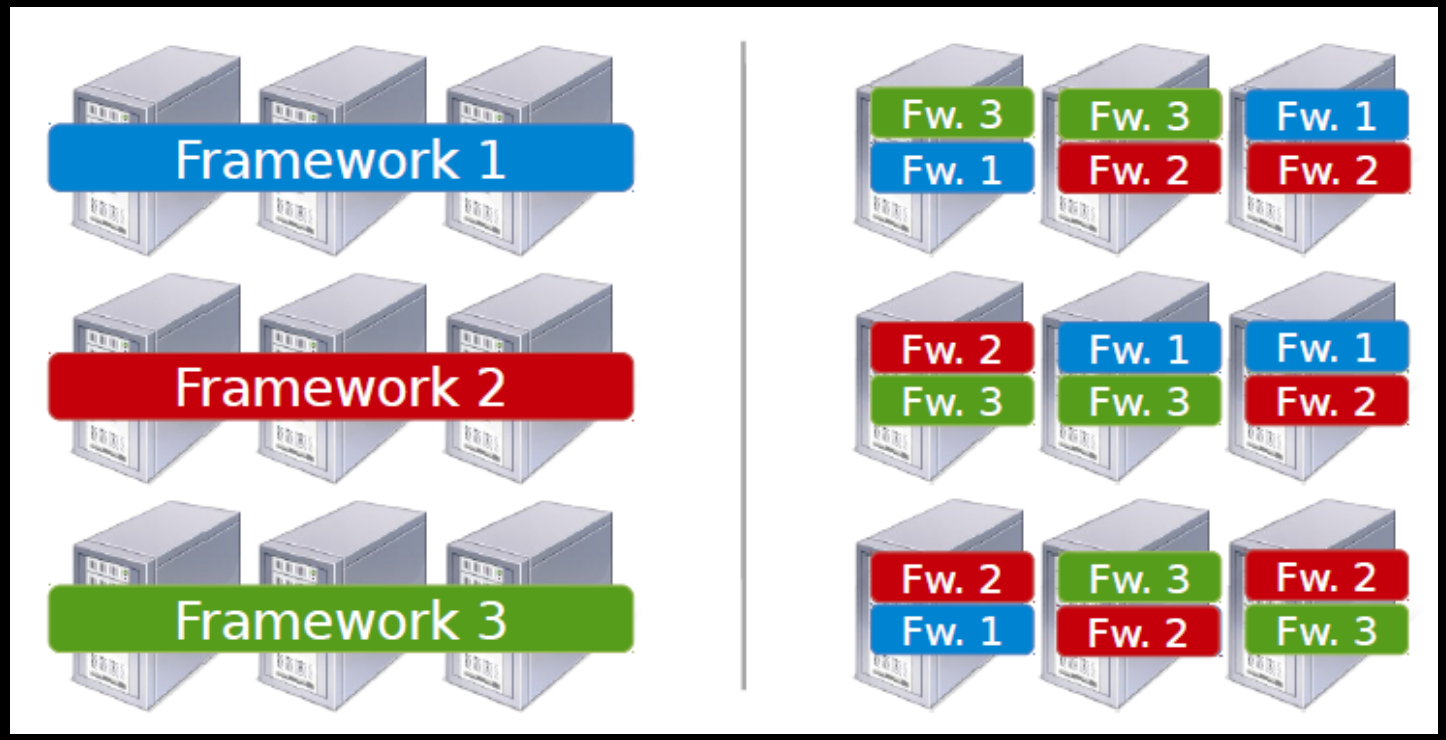
Two-Level Scheduler

- Advantages: simple and easy to extend
- Disadvantages: may lose something in terms of optimality

Mesos


Hadoop
Node
Node
Pregel
Node
Node
...
Hadoop
Pregel
...
Mesos
Node
Node
Node
Node

Fine-Grained Sharing



Coarse-grained sharing

Fine-grained sharing

Mesos Scheduler

- Mesos scheduler sends resource offers to frameworks
- Frameworks select which offers to accept and which tasks to run
- Unit of allocation: resource offer
  - Vector of available resources on a node

Mesos Scheduler: Example

1. Node 1 sends message to Mesos scheduler: <node 1, 4cpu, 4GB> (nodes periodically send status update to Mesos scheduler)
2. Mesos scheduler sends resource offer to a framework, for example Framework 1: <node 1, 4cpu, 4GB>
3. Framework 1 responds with either rejecting offer or accepting with a task list to be run: <task1, node1, 2cpu, 1GB>; <task2, node1, 1cpu, 1GB> (these choices made according to framework scheduler's rules)
4. Resulting tasks sent to nodes

Main Question

How should the Mesos scheduler decide which frameworks to make offers to?

Single Resource – Fair Sharing

1. Suppose that n users want to shar a single resource, say CPU
   - Allocate each $1/n$ of the shared resource
2. If a user wants less than its fair share, then generalized by max-min fairness:
   - Suppose there are three users, under fair sharing each gets 33% of the resource
   - If one user only wants 20%, then that user gets 20%; the remaining users get 40%
3. Can generalize with weights to reflect importance.
   - For example, two users, one with weight 2, the other with weight 1: first user gets 66%, the second user gets 33%

Properties of Max-Min Fairness

- Share guarantee
  - Each user can get at least $1/n$ of the resource
  - Will get less if demand is less
- Strategy proof
  - Users are not better off by asking for more than they need
  - Users have no reason to lie
- When is this not enough?
  - Multiple resources!

Multi-resource example

- 2 resources, CPUs and memory
- User 1 wants <1 CPU, 4GB> per task
- User 2 wants <2 CPU, 1GB> per task
- What is a fair allocation?

A Natural Policy

- Asset fairness: give weights to resources (e.g. 1 CPU = 1 GB) and equalize total value given to each user
- Total resources: 28 CPU and 56 GB RAM (so here, a natural choice is 1 CPU = 2 GB)
  - User 1 wants <1 CPU, 2 GB> per task
  - User 2 wants <1 CPU, 4 GB> per task
- Asset fairness yields the following problem:

$$\max(x, y)$$
$$x + y \leq 28$$
$$2x + 4y \leq 56$$
$$2x = 3y$$

- Result gives $x = 12$ and $y = 8$, so User 1 gets 43% of the CPUs and 43% of the RAM; User 2 gets 28% of the CPUs and 57% of the RAM

A Natural Policy

- Problem: User 1 gets less than 50% of both CPU and RAM, so would be better off in a separate cluster with half the resources

Dominant Resource Fairness (DRF)

- Dominant resource of a user: the resource that user has the biggest share of:
  - Total resources: <8 CPU, 5 GB>
  - User 1 allocation: <2 CPU, 1 GB>: 2/8=25% CPU and 1/5=20% RAM
  - Dominant resource of User 1 is CPU (25% > 20%)
- Dominant share of a user: the fraction of the dominant resource they are allocated
  - User 1 dominant share is 25%

Dominant Resource Fairness (DRF)

- Apply max-min fairness to dominant shares: give every user an equal share of their dominant resource
- Example
  - total resources <9 CPU, 18GB>
  - User 1 tasks need <1 CPU, 4GB>; dominant resource RAM (1/9 < 4/18)
  - User 2 tasks need <3 CPU, 1GB>; dominant resource CPU (3/9 > 1/18)
- Solve the following problem

$$\max(x, y)$$
$$x + 3y \leq 9$$
$$4x + y \leq 18$$
$$\frac{4x}{18} = \frac{3y}{9}$$

- Gives $x = 3$ (33% CPU, 66% RAM) and $y = 2$ (66% CPU, 16% RAM)

Where is (was) Mesos used?

- Twitter (since moved to Kubernetes)
- Netflix (since moved to Kubernetes)
- Mesos quite popular until a few years ago (arrival of Kubernetes)
- Two key issues: scaling and storage